service that maintains state, perhaps in memory, perhaps on disk. For example, a Redis service could be the second service that manages state.

The problem with a stateless service that stores data directly in the database is that it is not scalable. Each instance of the service needs a connection to the database to execute its logic. This makes the database a bottleneck or single point of failure. Each instance of the service competes for this resource, and if you try to scale your services tenfold, your database may not be able to scale as much, for example, in terms of the number of connections. If you decide to partition the database, finding the right sharding is anything but trivial. Stateless services only exchange some problems for others.

ATOMICITY

In distributed systems, *transactions* pose the problem of atomicity (the *A* of ACID). It can also be thought of as the possibility of aborting so either the transaction succeeds completely or nothing changes. Let’s focus here on what happens when a transaction involves multiple databases. For example, service T coordinates the booking of a trip and calls service C, which rents a car, and service F, which buys a flight. The services for the car and the flight can fail independently, so you may end up renting the car but not buying the flight. You can see this service dependency in figure B.5.